ASSIGNMENT NO. 03

FILE – 1 : Utils.py

```
import os

os.environ["TOKENIZERS_PARALLELISM"] = "false"
os.environ["HF_HUB_DISABLE_TELEMETRY"] = "1"
os.environ["TRANSFORMERS_NO_ADVISORY_WARNINGS"] = "true"

from transformers.utils import logging
logging.set_verbosity_error()

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from sentence_transformers import SentenceTransformer
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

stop_words = set(stopwords.words('english'))

def jaccard_details(s1, s2):
    words1 = set(word_tokenize(s1.lower())) - stop_words
    words2 = set(word_tokenize(s2.lower())) - stop_words

    intersection = words1.intersection(words2)
    union = words1.union(words2)

    score = len(intersection) / len(union)

    return words1, words2, intersection, union, score

def tfidf_cosine_details(s1, s2):
    vectorizer = TfidfVectorizer()
    tfidf_matrix = vectorizer.fit_transform([s1, s2])

    cosine_sim = cosine_similarity(tfidf_matrix[0:1], tfidf_matrix[1:2])[0][0]

    feature_names = vectorizer.get_feature_names_out()
    vec1 = tfidf_matrix.toarray()[0]
    vec2 = tfidf_matrix.toarray()[1]

    return feature_names, vec1, vec2, cosine_sim

def bert_similarity(s1, s2):
```

```
    model = SentenceTransformer('all-MiniLM-L6-v2')
    embeddings = model.encode([s1, s2])

    score = cosine_similarity([embeddings[0]], [embeddings[1]])[0][0]
    return score
```

File – 2:Main.py

```
from utils import jaccard_details, tfidf_cosine_details, bert_similarity

print("\n========= TEXT SIMILARITY ANALYZER =========\n")

text1 = input("Enter First Sentence  : ")
text2 = input("Enter Second Sentence : ")

print("\n--------------------------------------------")
print("Sentence 1:", text1)
print("Sentence 2:", text2)
print("--------------------------------------------")

words1, words2, inter, union, jaccard = jaccard_details(text1, text2)

print("\n ◇  JACCARD SIMILARITY ANALYSIS")
print("Words in Sentence 1:", words1)
print("Words in Sentence 2:", words2)
print("Common Words      :", inter)
print("Total Unique Words:", union)
print("Jaccard Score     :", round(jaccard, 3))

features, v1, v2, cosine = tfidf_cosine_details(text1, text2)

print("\n ◇  TF-IDF VECTORS")
print("\nFeatures:", list(features))

v1_clean = [float(x) for x in v1]
v2_clean = [float(x) for x in v2]

print("\nVector 1:", [round(x,3) for x in v1_clean])
print("Vector 2:", [round(x,3) for x in v2_clean])

print("\nCosine Similarity:", round(cosine, 3), f"({round(cosine*100,2)}%)")

bert = bert_similarity(text1, text2)

print("\n ◇  BERT SEMANTIC SIMILARITY")
print("Similarity Score:", round(bert, 3))
```

```python
print("\n============= FINAL ANALYSIS =============")

if bert > 0.75:
    meaning = "Both sentences have almost same meaning."
elif bert > 0.50:
    meaning = "Sentences are related in context."
elif bert > 0.25:
    meaning = "Sentences are slightly related."
else:
    meaning = "Sentences are different."

print("Conclusion:", meaning)
print("==========================================")
```

OUTPUT:

```
PS C:\Users\Rohit\Desktop\TextSimilarityLab\TextSimilarityLab> & C:/Users/Rohit/AppData/Local/Program
s/Python/Python313/python.exe c:/Users/Rohit/Desktop/TextSimilarityLab/TextSimilarityLab/main.py

========= TEXT SIMILARITY ANALYZER =========

Enter First Sentence  : The cat is sleeping on the sofa
Enter Second Sentence : A dog is resting on the couch

------------------------------------------
Sentence 1: The cat is sleeping on the sofa
Sentence 2: A dog is resting on the couch
------------------------------------------

 ◆ JACCARD SIMILARITY ANALYSIS
Words in Sentence 1: {'sofa', 'cat', 'sleeping'}
Words in Sentence 2: {'dog', 'couch', 'resting'}
Common Words      : set()
Total Unique Words: {'dog', 'couch', 'sleeping', 'sofa', 'resting', 'cat'}
Jaccard Score     : 0.0

 ◆ TF-IDF VECTORS

Features: ['cat', 'couch', 'dog', 'is', 'on', 'resting', 'sleeping', 'sofa', 'the']

Vector 1: [0.407, 0.0, 0.0, 0.29, 0.29, 0.0, 0.407, 0.407, 0.579]
Vector 2: [0.0, 0.47, 0.47, 0.335, 0.335, 0.47, 0.0, 0.0, 0.335]

Cosine Similarity: 0.388 (38.77%)
modules.json: 100%|| 349/349 [00:00<00:00, 1.16MB/s]
config_sentence_transformers.json: 100%|| 116/116 [00:00<00:00, 522kB/s]
README.md: 10.5kB [00:00, 16.1MB/s]
sentence_bert_config.json: 100%|| 53.0/53.0 [00:00<00:00, 55.3kB/s]
config.json: 100%|| 612/612 [00:00<00:00, 2.99MB/s]
model.safetensors: 100%|| 90.9M/90.9M [00:17<00:00, 5.19MB/s]
Loading weights: 100%|| 103/103 [00:00<00:00, 1740.77it/s, Materializing param=pooler.dense.weight]
tokenizer_config.json: 100%|| 350/350 [00:00<00:00, 1.23MB/s]
vocab.txt: 232kB [00:00, 15.2MB/s]
tokenizer.json: 466kB [00:00, 35.3MB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable high
er rate limits and faster downloads.
special_tokens_map.json: 100%|| 112/112 [00:00<00:00, 326kB/s]
```

```
 ◆ BERT SEMANTIC SIMILARITY
Similarity Score: 0.486

============= FINAL ANALYSIS =============
Conclusion: Sentences are slightly related.
==========================================
PS C:\Users\Rohit\Desktop\TextSimilarityLab\TextSimilarityLab>
```